Indexing for Performance

Kimberly L. Tripp
she / her
President / Founder
SQLskills.com

Kimberly L. Tripp

she/her

President / Founder

SQLskills
immerse yourself in sql server

@KimberlyLTripp

Kimberly@SQLskills.com

https://www.SQLskills.com

https://www.SQLskills.com/blogs/kimberly

Consultant / Trainer / Speaker / Writer

- Author / instructor for SQL Server Immersion Events: IEPTO1, IEPTO2, IEVLT, IETLB, IEQuery, and IEHADR
- Author / presenter for Pluralsight
- Instructor and exam author for SQL Server and Sharepoint MCMs
- Author / manager of SQL Server 2005 and 2008 Launch Content
- Author / speaker at Microsoft TechEd, SQLPASS, ITForum, TechDays, …
- Author of SQL Server Whitepapers on MSDN/TechNet: Partitioning, Snapshot Isolation, Manageability, SQLCLR for DBAs
- Author / presenter for MSDN and TechNet (25+)
- Instructor / SME for Microsoft University

Data Platform MVP

- 17 years: 2002-2019

When I'm not working with SQL

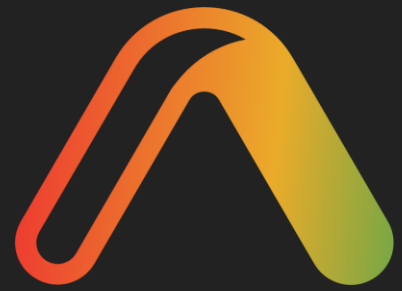
- Traveler / adventurer and avid diver / photographer: www.BlueWaterImages.com
- @KimberlyLTripp on Insta

Blue Water IMAGES

PASS Data Community SUMMIT

Materials

- Session demo zip and slides will be posted AFTER my session
- Email if you have problems finding it or opening it from Friday (pm) onward
  - Kimberly@SQLskills.com
  - Paul@SQLskills.com



PASS
Data Community
SUMMIT
2023

Agenda

- Indexing for performance
- Are you ready to add indexes
- Indexing strategies
- Query tuning
- Server tuning

Indexing for Performance

Is NOT only about creating indexes…

- It's about making sure they're going to get used
  - ❑ Are you getting a cached plan?
  - ❑ Do you have seekable expressions?
  - ❑ *And, no, I'm NOT talking about forcing – don't do that!*
- It's about making sure the data types on which they're defined are efficient
  - ❑ Are your data types unnecessarily wide?
  - ❑ Do you have a well-chosen clustering key?
- It's about making sure your tables / joins are efficient
  - ❑ Are you joining on integers / numerics or large character strings / GUIDs?

Indexing for Performance

Is NOT only about creating indexes…

- It's about asking better questions
  - Selection: asking only for the rows you need and with a selective (SEEKABLE) expression
    - Isolated the column on one side of the expression:
      - WHERE monthlysalary * 12 > literal ← Non-seekable
      - WHERE monthlysalary > literal / 12 ← Seekable
  - Projection: asking only for the columns you need AND NO MORE
  - And so, so, so many more Transact-SQL optimizations
- It's about having better information
  - Do you have up-to-date statistics?
  - Do you have the right statistics?
- There's no magic / single thing that's going to make your servers super FAST
  - Indexing for Performance is about finding the right balance between too many and too few indexes

EVERY SINGLE QUERY HAS A PERFECT INDEX...

- But, you can't possibly index EVERY single query
- And, you shouldn't try!
- You need to index like the optimizer optimizes
  - ❑ It does not strive to find the best plan
  - ❑ It finds a GOOD PLAN FAST...
- If you're ready to "add" indexes – How?
  - ❑ Step 1: Query Tuning
  - ❑ Step 2: Server Tuning

Are You Ready to Add Indexes?

NOTE: Check out the hidden slides with more information!

- Probably not…
- BEFORE you add more indexes
  1. Index cleanup (get rid of any dead weight)
     - ❑ Remove duplicate indexes
     - ❑ Remove unused indexes (consider disabling / testing / then dropping)
  2. Index health
     - ❑ Determine health of existing (and useful) indexes
  3. Statistics health
     - ❑ Determine health of existing (and possibly adding more) statistics
  4. Now, you're ready…
     - ❑ Slowly and iteratively – add needed indexes

(1) Remove Unused Indexes

hidden slide w/extra details

DMVs Don't Tell You Everything…

- Removing redundant / duplicate indexes
- Duplicate indexes can still show as used
- *MUST* review your indexes
  - ❑ Don't forget INCLUDEd columns (in 2005) or filtered indexes (in 2008)
  - ❑ sp_helpindex doesn't show these columns
  - ❑ Use my updated version of sp_SQLskills_helpindex to get better information and determine if one index really is redundant/duplicate
  - ❑ Check out the resource scripts for both of these! sp_SQLskills_finddupes
- Don't rely on sys.dm_db_index_usage_stats alone

(1) Dropping an Index

hidden slide w/extra details

- What Could Go Wrong?
  - ❑ Queries that use index hints will ERROR if the index no longer exists
    - ❑ This is the reason for why SQL Server allows duplicate indexes to be created in general…
  - ❑ Plans guide might no longer work
    - ❑ They're just invalidated, a new plan will be used
  - ❑ Plans could change
    - ❑ This could be good… or bad?!

(2) Verify Health of Existing Indexes…

hidden slide w/extra details

- Some think that fragmentation is NOT an issue because of solid state drives
- It is STILL an issue:
  - ❑ Fragmentation is caused by splits – splits incur overhead in logging / cache – adding time and wasting space
- Addressing fragmentation can be difficult – many considerations:
  - ❑ Table size and usage patterns
  - ❑ Impact to availability – can you use an online operation?
    - ❑ ALTER INDEX…REBUILD…WITH (ONLINE = ON)
      - ❑ Not if the index has a LOB column in it (fixed in 2012) but NOT for legacy LOBs (text, ntext, image)
      - ❑ Not if you try to rebuild only a single partition (fixed in 2014)
    - ❑ ALTER INDEX…REORGANIZE (always online)

(2) Verify Health of Existing Indexes…

- ❑ Reorganizing always uses 'full' logging but the amount of log information generated will depend on how much fragmentation exists
- ❑ There are trade-offs between rebuilding and reorganizing in terms of log space, disk space, run-time, impact to tempdb and even benefit…
- ❑ Resources
  - ❑ Whitepaper: Microsoft SQL Server 2000 Index Defragmentation Best Practices (concepts still apply)
  - ❑ Whitepaper: Online Indexing Operations in SQL Server 2005 (if you're interested in how the ONLINE operations work, concepts still apply)

- Paul's Pluralsight course: SQL Server: Index Fragmentation Internals, Analysis, and Solutions

Indexing for Performance

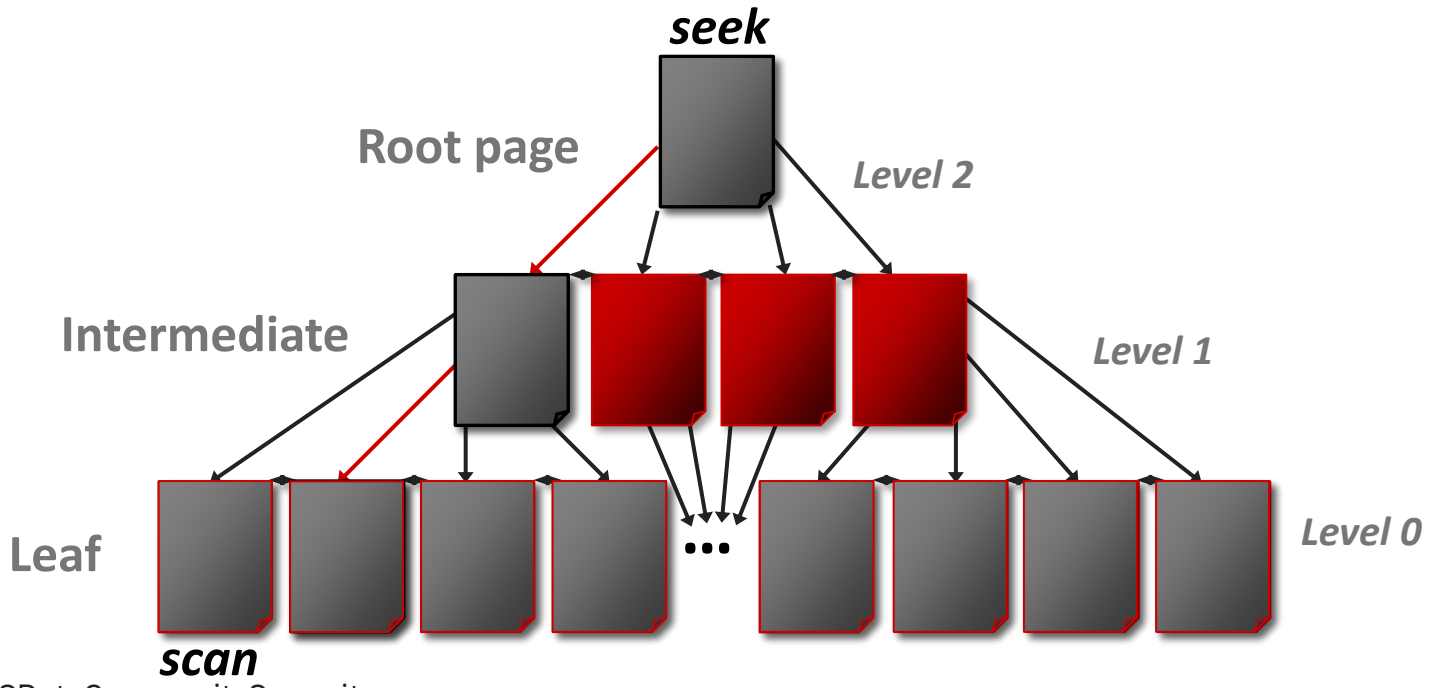
- Indexing strategies
  - ❑ Row-based indexes
    - ❑ Traditionally our only choice
    - ❑ Excellent for mixed workloads (OLTP or OLTP/limited decision support)
  - ❑ Column-based indexes
    - ❑ AMAZING for relational data warehousing
    - ❑ EXCELLENT for large scale analysis, aggregations
- Do you still need indexes views?
  - ❑ Columnstore indexes have fewer requirements
  - ❑ If you aggregate down to a very small set (sum(sales) by county) these might be a HUGE win for READ-ONLY data sets (can be ideal with partitioned views)

Indexing for Performance

- Indexing strategies
  - Row-based indexes
    - Traditionally our only choice
    - Excellent for mixed workloads (OLTP or OLTP/limited decision support)
  - Column-based indexes
    - AMAZING for relational data warehousing
    - EXCELLENT for large scale analysis, aggregations
- Do you still need indexed views?
  - Columnstore indexes have WAY FEWER requirements
  - If you aggregate down to a very small set (sum(sales) by county) indexed views might be a HUGE win for READ-ONLY data sets (can be ideal with partitioned views)

Index Structures: Row-based Indexes

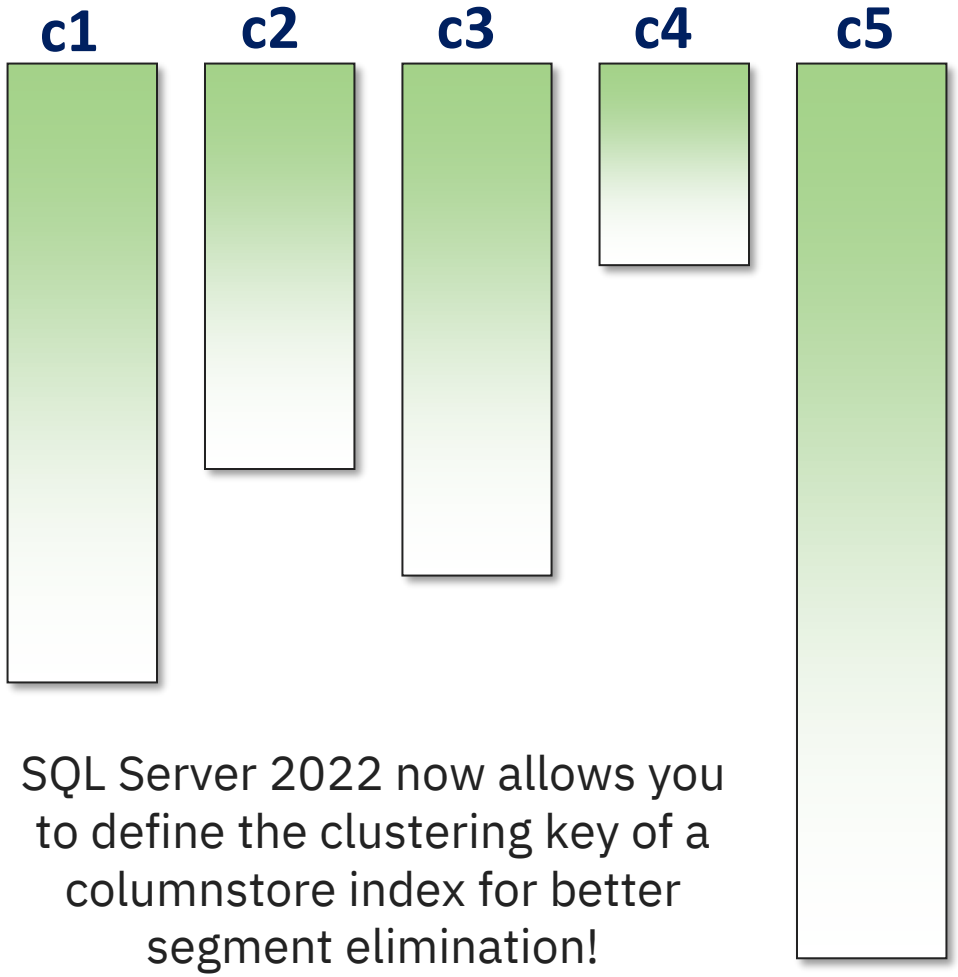
- Leaf level: contains something for every row of the table in indexed order
- Non-leaf level(s) or B-tree: contains something, specifically representing the FIRST value, from every page of the level below. Always at least one non-leaf level. If only one, then it's the root and only one page. Intermediate levels are not a certainty.



- Seekable for any left-based subset of the key
- Scanable for any combination of the columns in the index

Index Structures: Column-based Indexes

- Column-based storage
- Only that column is stored on the same page – potentially HIGHLY compressible!
- Data is segmented for better batch processing
- SQL Server can do "segment elimination" (similar to partition elimination) and further reduce the number of batch(es) to process!
- Best place for info: http://www.nikoport.com/columnstore/



SQL Server 2022 now allows you to define the clustering key of a columnstore index for better segment elimination!

hidden slide w/extra details

Row-based Indexes v. Column-based Indexes

Row-based indexes

- Supports two forms of data compression
  - Row compressed
  - Page compressed
- Effective point queries / seeks (NC)
- Wide variety of supported scans
  - Full / partial table scans (CL)
  - Nonclustered covering scans (NC)
  - Nonclustered covering seeks with partial scans (NC)
- Biggest problems
  - Must create the appropriate indexes
  - Must store the data (not as easily compressed)

Column-based indexes

- Significantly better compression
  - COLUMNSTORE / COLUMNSTORE_ARCHIVE
- Supports large scale aggregations
- Support partial scans w/"segment" elimination
  - Only the needed columns are scanned
  - Data is broken down into row groups (roughly 1M rows) and segments can be eliminated
  - Combine w/partitioning for further elimination
  - Parallelization through batch mode processing
- Biggest problems
  - Minimum set for reads is a row group (no seeks)
  - Limitations of features for batch mode by version
  - Limitations with other features (less and less by SQL Server version)

Indexing for Performance

- Query Tuning
  - ❑ Requires THREE things:
    - ❑ Knowing your data
    - ❑ Knowing your workload
    - ❑ Knowing how SQL Server works
  - ❑ Should you even tune this query? Is it critical?
  - ❑ Is its cumulative cost worth tuning?
  - ❑ If so, find the best index for the query… DO NOT IMPLEMENT IT in production!
- Server Tuning
  - ❑ Take the best index that the query needs and make sure it's really what the server needs!

Query Tuning: Row-store Indexes

- You do NOT always have to have the BEST index…
  - ❑ SQL Server uses the best index available
  - ❑ Tuning that you do for critical / frequently executed queries will very likely help other queries
  - ❑ PICK YOUR BATTLES / KNOW YOUR WORKLOAD!
    - ❑ Review sys.dm_exec_query_stats
    - ❑ Review sys.dm_exec_procedure_stats

- Once you know the queries to tune:
  - ❑ Review the green hint
  - ❑ Consider using DTA for isolated query tuning
  - ❑ Learn internals, manual strategies, other tricks, etc…

Benefits of These Indexes?

hidden slide w/extra details

- The green hint/the Missing Index DMVs recommendation:
  - ❑ Leads with the column charge_amt
  - ❑ This removes the table scan and changes to an index seek
  - ❑ This allows filtering by our search argument (charge_amt > 2500)
  - ❑ PRO: This helps tune the plan that was chosen/executed
  - ❑ CON: They did not hypothesize for alternatives
- The DTA's recommendation:
  - ❑ Is mostly the same as the green hint (in this case)
  - ❑ ADDs all key columns to the key (which we don't need)
  - ❑ Might be different – if so, *definitely* test it; it might be BETTER
  - ❑ PRO: they do a deeper analysis of the query than the query hint does!
- Of the tools – what's better? What gives better performance?
  - ❑ Usually, DTA (either the same or sometimes better!)

Database [Engine] Tuning Advisor

hidden slide w/extra details

- It's not always perfect
- It sometimes yields the same index recommendation as the missing index DMVs
- It often OVER recommends indexes (this is why you want to use it ITERATIVELY after really analyzing where to begin)
- It doesn't recommend any form of index consolidation
  - ❑ This is one of the reasons that a lot of development environments end up over-indexed
- IF you end up creating the index that was recommended for a particular table, then create the statistics that are recommended for that table
  - ❑ DTA can create multi-column statistics
  - ❑ The can give the optimizer other (sometimes VERY useful) ways of using the recommended index!

Query Tuning in Preparation for Server Tuning

- The index definition should always be ONLY what the specific query needs
- Do not "add the clustering key" because you know that SQL Server will
  - ❑ No, SQL Server won't add it twice in the index
  - ❑ Later, you might not remember that it's not *required* for that query
    - ❑ Which could limit your consolidation options

Scenario

Query tuning wants this index:

```
CREATE INDEX [NCIndexLNinKeyInclude3OtherCols] ON [dbo].[member] ([lastname])
INCLUDE ([firstname], [middleinitial], [phone_no]);
```

Internally, SQL Server creates a structure that ADDs the clustering key (in the tree when it's non-unique)

```
[dbo].[member] ([lastname], [member_no])
INCLUDE ([firstname], [middleinitial], [phone_no]);
```

But, if you were to create this index – I'd think that member_no were NEEDED in the key

(see, the Index Consolidation Slide for more details)

Query Tuning: Too Many Cooks in the Kitchen!

- You find a query that needs indexes…
- Your colleagues find queries that need indexes…
- ITW/DTA find queries that need indexes…
- The missing index DMVs find queries that need indexes…

- We think these tools/people are helping – and the indexes ***definitely*** help queries (they're not wrong)
- But, you may end up with a lot of similar indexes
  (hopefully only similar – and not identical? – indexes)

SQL Server will let you create as many useless indexes as you like…

Index Consolidation for *Better* Covering

- Index structures
  - Key is used for navigation
    - Must preserve the left-based seeking capability of the key
  - INCLUDE is for covering
    - Order of the columns in the include is irrelevant
- Imagine the following indexes:
  - Ind1: (Lastname, Firstname, MiddleInitial)
  - Ind2: (Lastname) INCLUDE (Phone)
  - Ind3: (Lastname, Firstname) INCLUDE (SSN)
- COMBINE all three:
  - (Lastname, Firstname, MiddleInitial) INCLUDE (Phone, SSN)
- Before you create ANY new indexes, review the current indexes!

Server Tuning: The RIGHT Way to a Healthier Server

- Not necessarily the best index for the query
- But, a better overall index for the server:
  - ❑ Start with the query's best index
  - ❑ Review existing indexes
    - ❑ Are there any similar?
    - ❑ Index consolidation!
  - ❑ Review the missing index DMVs
    - ❑ Any additional indexes with which you can consolidate?
- Test the new "server tuning" index:
  - ❑ Create the new index
  - ❑ Disable the redundant / consolidated indexes (once proven, drop them)

Index Consolidation Considerations

hidden slide w/extra details

- Consolidating existing indexes is not quite as simple as removing all left-based subsets:
  - ❑ It's true that a query using an index on LastName alone COULD use an index on LastName, Firstname OR an index on LastName, Firstname, MI but, always be careful of how much larger the indexes are and the type of queries using them
  - ❑ Should you drop indexes that are left-based subsets of others?
    - ❑ Typically yes BUT, consider the width of the additional columns
    - ❑ While any left-based subset CAN be handled by a larger superset, should it?
    - ❑ If the additional column(s) are relatively wide and only needed for a couple of queries whereas the narrower version is used by a lot of queries, you might want both!

Indexing for Performance

- Is NOT just about indexes
  - ❑ Coding best practices
  - ❑ Statistics best practices
  - ❑ Maintenance best practices
- It's about BALANCE
- Start with "query tuning"
- Implement "server tuning" strategies for true success
- Repeat!

What to do next?

- Lots of content this week…
  - ❑ Reviewing sooner rather than later will help!
  - ❑ Using a mix of mediums will help you see things in different ways and likely for some of you one will be better than another – multiple will be the way to make it stick!
- A possibly review strategy?
  - ❑ Review the slides and course resources
  - ❑ Watch the recommended videos
  - ❑ Offer some "short courses" or "brown bag" lunch sessions where you deliver 30-45 minute topics to team members

    If you want to learn something, read about it.
    If you want to understand something, write about it.
    If you want to master something, teach it. – Yogi Bhajan
  - ❑ Always keep learning!

PLURALSIGHT

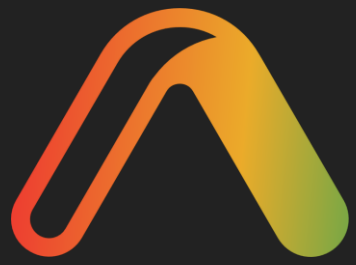
- SQL Server: Why Physical Database Design Matters
- SQL Server: Indexing for Performance
- SQL Server: Optimizing Ad Hoc Statement Performance
  - ❑ Just over 7 hours on ad hoc statement execution and plan caching
- SQL Server: Optimizing Stored Procedure Performance (Part 1)
  - ❑ Just over 7 hours on procedure caching and recompilation
- SQL Server: Optimizing Stored Procedure Performance – Part 2
  - ❑ Just under 4 hours on session settings and caching

Thank you

May all your queries have up-to-date statistics, useful covering indexes, and appropriately cached plans!

Kimberly L. Tripp
she / her
President / Founder
SQLskills.com

SQLskills
immerse yourself in sql server



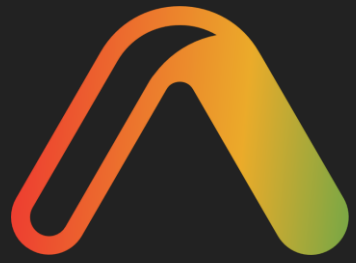
PASS
Data Community
SUMMIT
2023

Session evaluation

Your feedback is important to us

Evaluate this session at:

www.PASSDataCommunitySummit.com/evaluation



PASS
Data Community
SUMMIT
2023